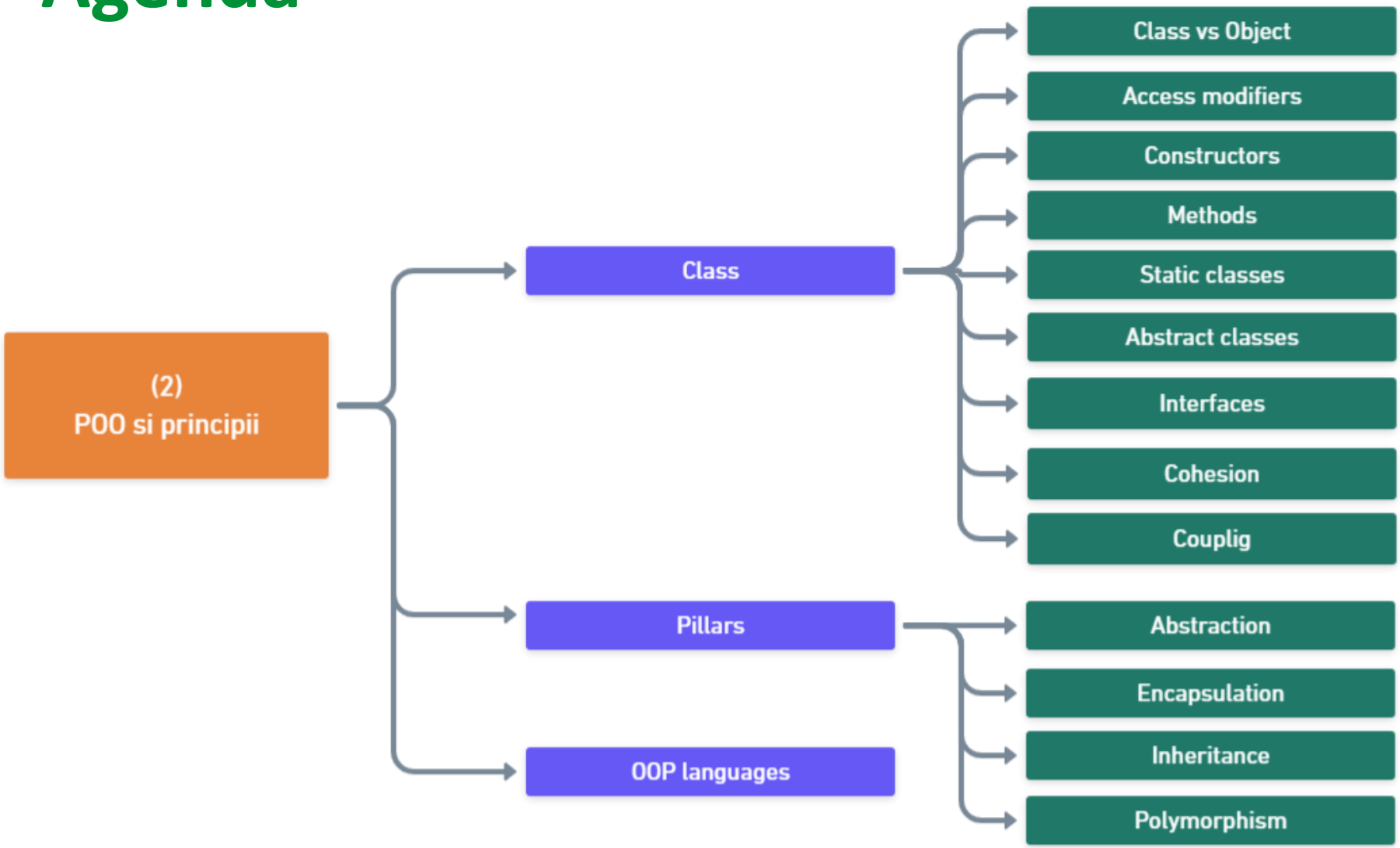
POO si PRINCIPII

Tehnici avansate de programare

Nicoleta Scrimint

Agenda

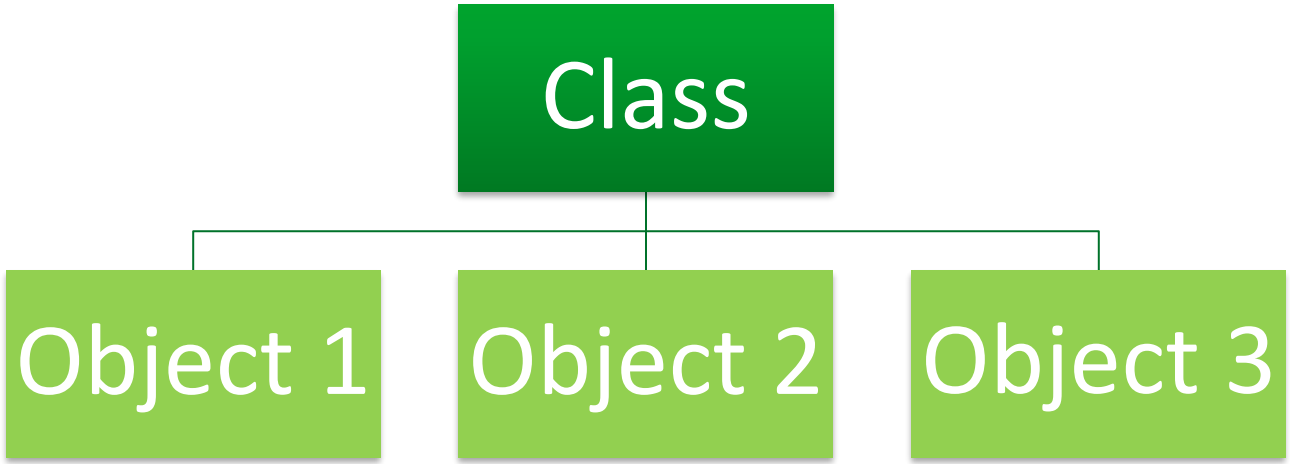


Class

- Class is a user-defined blueprint or prototype of an object that describes how that specific object will look like.
- User-defined type that model the concepts that are specific to the program's problem domain.

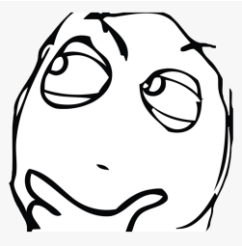
Class vs Object

- Object is a single instance of a class



Access modifiers

- **public** - can be accessed from anywhere
- **protected** - can only be accessed from the same class and derived classes
- **internal** - can only be accessed from the same project
- **private** - can only be accessed by other members from the same class
- **protected internal** - ???
- **private protected** - ???



Access modifiers

- **public** - can be accessed from anywhere
- **protected** - can only be accessed from the same class and derived classes
- **internal** - can only be accessed from the same project
- **private** - can only be accessed by other members from the same class
- **protected internal** - can only be accessed from the same assembly OR from the derived classes
- **private protected** - can only be accessed from the same class and derived classes, from same assembly

Constructors

- Usage: initializes the class
- Syntax
- Default constructors

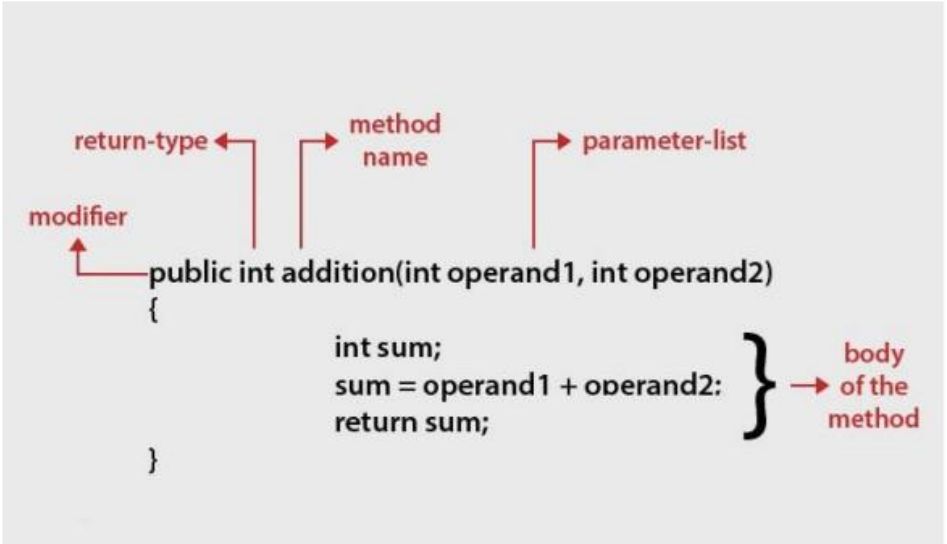
```
2 references
public class Student
{
    private string name;
    private string email;

    //default constructor
    0 references
    public Student()
    {
    }

    //constructor with parameters
    0 references
    public Student( string name, string email)
    {
        this.name = name;
        this.email = email;
    }
}
```

Methods

Usage: provides functionality to the clients of the class

Structure

- Signature
  - The access modifiers
  - The parameter modifiers
  - The return type
  - The name of the method
  - Any method parameters
- Method body



Static classes

Usage: We want some unchanging information to be available for being shared all the time when our application is running.

Properties:

- Defined using **static** keyword
- Can NOT be instantiated
- Contains only static members (properties and methods)
- Shares the lifetime of the application

Static classes

Definition

```
4 references
public static class MyCollege
{
    //static fields
    public static string CollegeName;
    public static string Address;

    //static constructor
    0 references
    static MyCollege()
    {
        CollegeName = "ABC College of Technology";
        Address = "Hyderabad";
    }

    //static method
    1 reference
    public static void Display()
    {
        Console.WriteLine("College Name: {0}, Address: {1}", CollegeName, Address);
    }
}
```

Usage

```
//Access static fields
Console.WriteLine(MyCollege.CollegeName);
Console.WriteLine(MyCollege.Address);


//Access static methods
MyCollege.Display();
```

```
//ERROR - Cannot create an instance of static class
var college = new MyCollege();
```

Abstract classes

- Defined using **abstract** keyword
- Can NOT be instantiated
- Contain at least one abstract member
- Are used to define base class in a hierarchy
- Require to be inherited

```
2 references
public abstract class Animal
{
    //abstract property, need to be overriden in inherited class
    2 references
    public abstract string Type { get; }

    //property
    2 references
    public string Name { get; }

    //abstract method, need to be overriden in inherited class
    2 references
    public abstract void Eat();


    //method with implementation
    0 references
    public void Sleep()
    {
        Console.WriteLine("Every animal sleep!");
    }

    //constructor
    1 reference
    protected Animal(string name)
    {
        Name = name;
    }
}
```

Abstract classes

Usage

```
//class that inherited abstract class
2 references
public class Dog : Animal
{
    //constructor that uses abstract class constructor
    1 reference
    public Dog(string name): base(name)
    {
    }

    //overriding property
    2 references
    public override string Type
    {
        get { return "dog"; }
    }

    //overriding method
    2 references
    public override void Eat()
    {
        Console.WriteLine("Dog eats!");
    }
}
```

```
//create instance of Dog class
var dog = new Dog( name: "Grivei");

//use overridden property
Console.WriteLine(dog.Type + dog.Name);

//use overridden method
dog.Eat();
```

```
//ERROR - Cannot create a instance of Abstract class
var cat = new Animal( name: "Tom");
```

Interfaces

Usage:

- It always helps if you can view it as a contract:
  - It defines the properties a class should have
  - It defines the actions a class should do, but it does not give you a way to do it
- Abstraction of different business concepts, e.g. output device: CD-ROM, USB
- Decouple from big impact third-party dependencies, e.g. payment system for a hotel reservation website
- Helps on testing the application by isolating the third party dependencies in the integration tests

Interfaces

- Defined using interface keyword
- Can NOT be instantiated
- Requires to be implemented
- Contain properties and methods
- Usually, methods don't have body
- Can contain methods with body since C# 8
- Can contain static methods since .Net 6

```
//interface
1 reference
public interface IAnimal
{
    //interface method(does not have a body)
    2 references
    void MakeSound();
}


//the Pig class "implements" the interface IAnimal
1 reference
public class Pig:IAnimal
{
    //the implementation of MakeSound method
    2 references
    public void MakeSound()
    {
        Console.WriteLine("The pigs say: wee wee!");
    }
}
```

```
//create class Pig
var pig = new Pig();

//use method MakeSound()
pig.MakeSound();
```

Interfaces

```
public static void Main()
{
    var cat = new Cat();
    var noise = cat.MakeNoise();
    Console.WriteLine("Cat noise: " + noise);
}

public interface IAnimal
{
    string MakeNoise();
}

public class Cat : IAnimal
{
    public string MakeNoise()
    {
        return "Meowww";
    }
}
```

```
public static void Main()
{
    var cat = new Cat();
    var noise = cat.MakeNoise();
    Console.WriteLine("Cat noise: " + noise);

    cat.DoFelineStuff();
}

public interface IAnimal
{
    string MakeNoise();
}

public interface IFeline
{
    void DoFelineStuff();
}

public class Cat : IAnimal, IFeline
{
    public string MakeNoise()
    {
        return "Meowww";
    }

    public void DoFelineStuff()
    {
        // TODO feline stuff
    }
}
```

Abstract or Interface

- It contains both declaration and implementation parts
- A class can only use one abstract class
- It is used to implement the core identity of class
- It contain constructor

- It contains both declaration and implementation parts (since C# 8)
- A class can use multiple interface.
- It is used to define a contract that classes must implement
- It does not contain constructor

Classes and objects in C# Knowledge check

- https://learn.microsoft.com/en-us/training/modules/create-classes-objects-c-sharp/
- Units:
  - 3 - Examine the .NET type system
  - 4 - Design and use classes
  - 7 - Module assessment

Cohesion

- Class cohesion is a measure of how closely related all the responsibilities, data, and functionality of a class are to each other.
- The functionality responsibilities can be bound based on how often, why they are changed, if they are changed together - information taken from business persons.

WHY?

- Cohesive classes are easier to understand, maintain, and test.
- Promotes reusability of code.
- Helps in reducing complexity and improving readability.
- Enhances the overall quality and maintainability of the software.

Cohesion - example

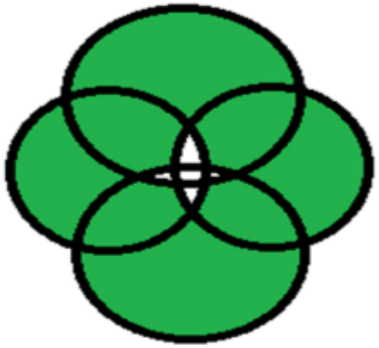
```
class Calculator {
    public int Add(int a, int b) {
        return a + b;
    }

    public int Subtract(int a, int b) {
        return a - b;
    }

    public int Multiply(int a, int b) {
        return a * b;
    }

    public double Divide(int a, int b) {
        if (b != 0) {
            return (double)a / b;
        }
        else {
            throw new DivideByZeroException("Cannot divide by zero");
        }
    }
}
```

Cohesion - issues

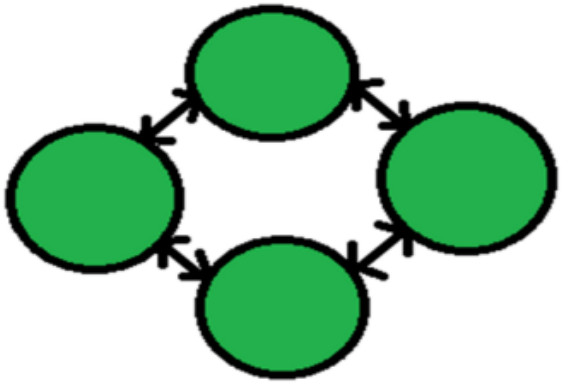
```
class Employee {
    public void CalculatePay() {
        // Calculates the pay of the employee
    }

    public void SendEmail() {
        // Sends an email to the employee
    }

    public void GenerateReport() {
        // Generates a report for the employee
    }
}
```

Coupling

Class coupling is a measure on how one class is connected or dependent with another class.



Tight coupling:
1. More Interdependency
2. More coordination
3. More information flow



Loose coupling:
1. Less Interdependency
2. Less coordination
3. Less information flow

Coupling

- Tight coupling:
  - Business functionality uses database logic specific for SQL Server database
  - Bind responsibilities which change on different times by different reasons, e.g. infrastructure like authentication and business logic
- Low coupling:
  - Use interfaces to communicate with database logic from the business logic
  - Isolate database logic from business domain classes
  - Use Middlewares for authentication

```
public class FileProcessor
```

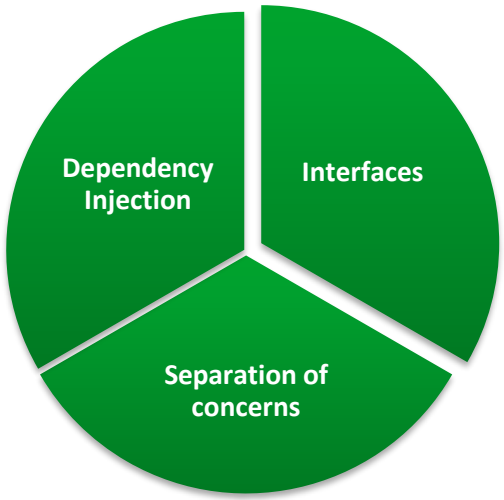
```
public interface IFileProcessor
```

Loose Coupling

WHY?

- **Flexibility**: Easier to modify and extend without impacting other parts of the system.
- **Testability**: Facilitates unit testing since components can be tested independently.
- **Maintainability**: Changes in one module have minimal impact on other modules.

HOW?



POO Pillars

Abstraction

Encapsulation

Inheritance

Polymorphism

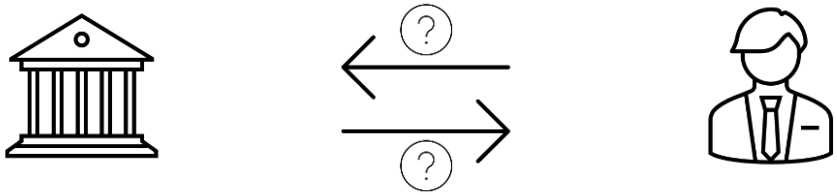
Abstraction

Abstraction

Definition

- Abstraction refers to modeling the relevant attributes and interactions of entities as classes to define an abstract representation of a system

Example



Key Points

- Abstraction allows developers to work with high-level concepts and reduces complexity by focusing on essential features.

Abstraction

```
public interface IAccount {
    void Deposit(decimal amount);
    void Withdraw(decimal amount);
}

public class SavingsAccount : IAccount {
    public void Deposit(decimal amount) {
        // Deposit implementation
    }

    public void Withdraw(decimal amount) {
        // Withdraw implementation
    }
}
```

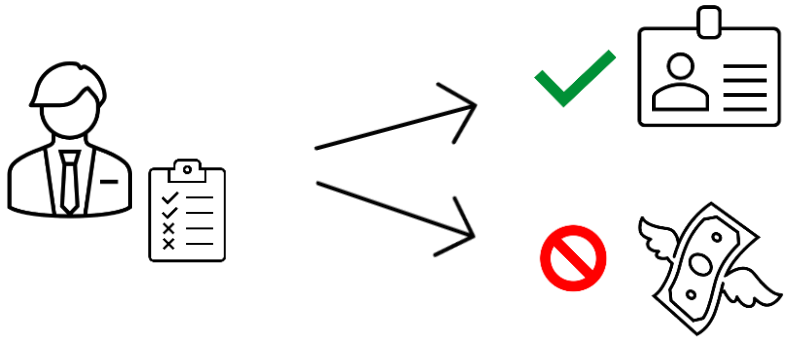
Encapsulation

Encapsulation

Definition

- Encapsulation refers to hiding the internal state and functionality of an object and only allowing access through a public set of functions

Example

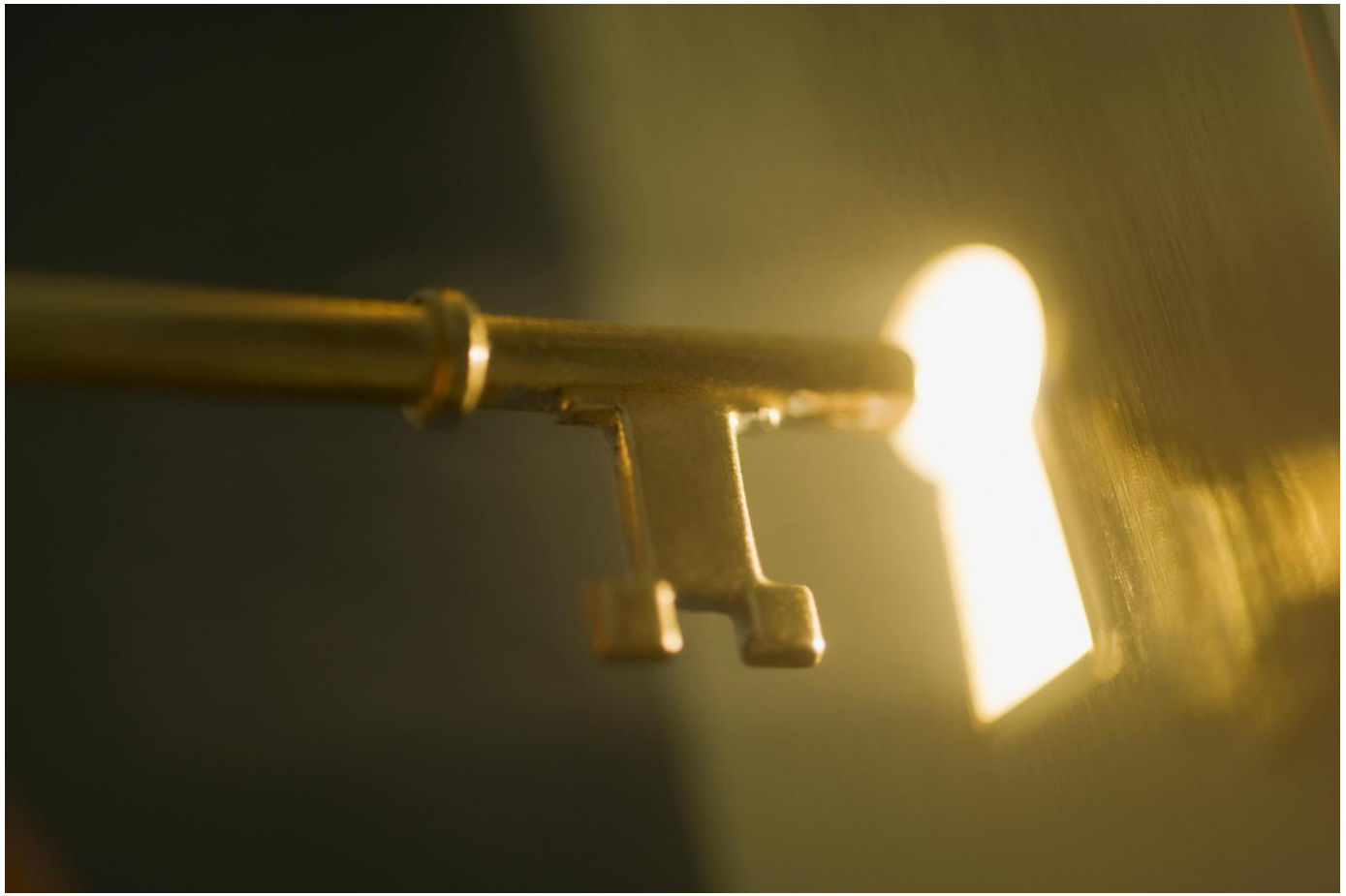


- BankAccount class provides public methods to handle money balance

Encapsulation

```
public class Employee {
    private string name;
    private int age;

    // Encapsulated properties
    public string Name {
        get { return name; }
        set { name = value; }
    }

    public int Age {
        get { return age; }
        set { age = value; }
    }

    // Encapsulated method
    public void DisplayDetails() {
        Console.WriteLine($"Name: {name}, Age: {age}");
    }
}
```

Inheritance

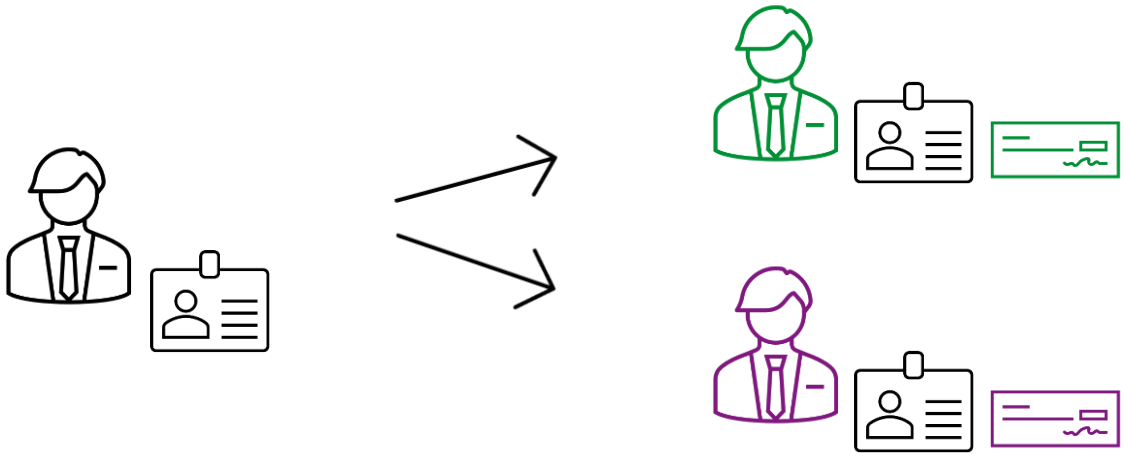


Inheritance

Definition

- Inheritance allows a class (subclass or derived class) to inherit the properties and methods of another class (superclass or base class).

Example



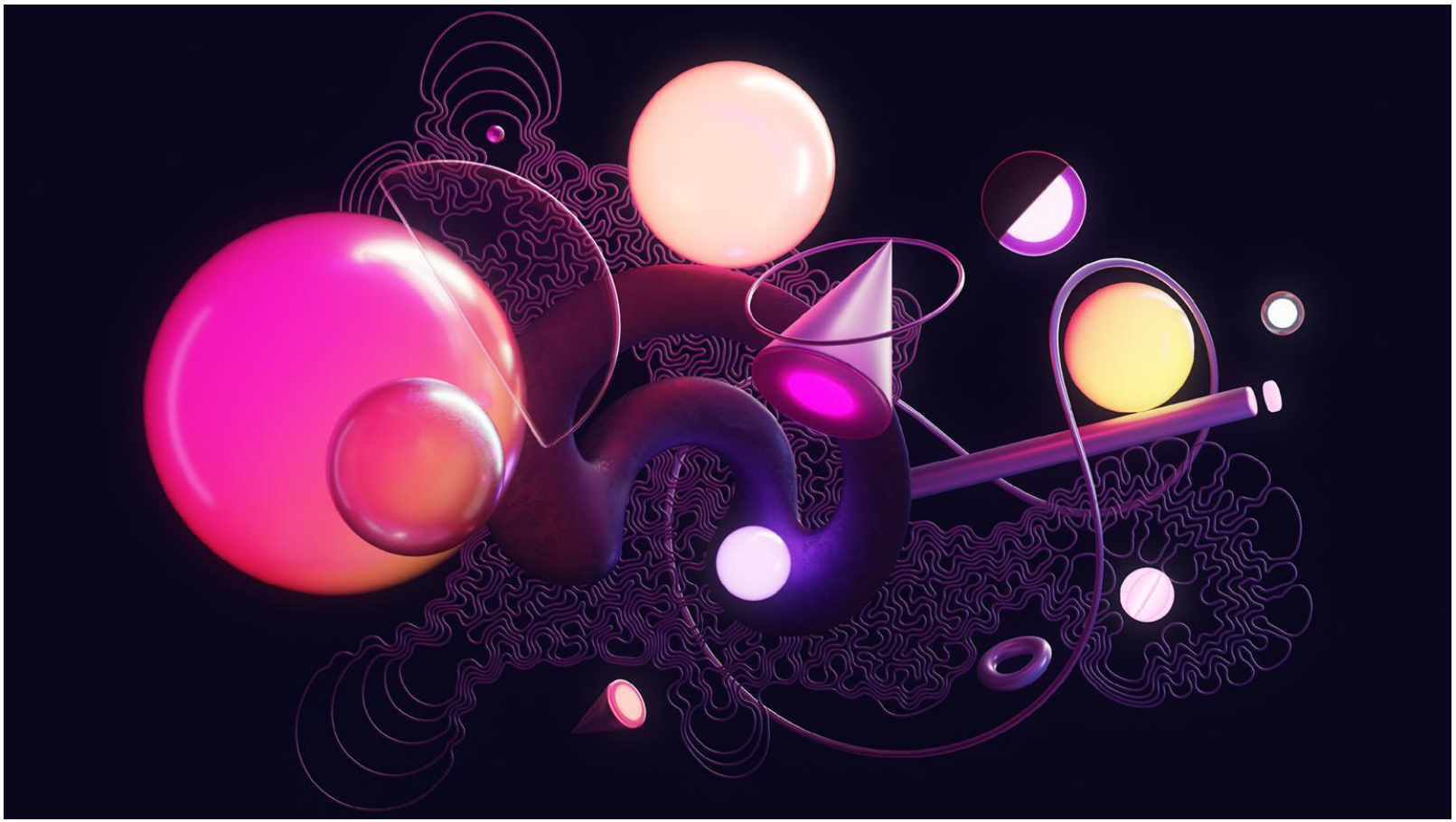
Key Points

- Inheritance promotes code reusability and establishes a hierarchical relationship between classes.

Inheritance

```
public class Animal {
    public void Eat() {
        Console.WriteLine("Animal is eating");
    }
}

public class Dog : Animal {
    public void Bark() {
        Console.WriteLine("Dog is barking");
    }
}
```

Polymorphism

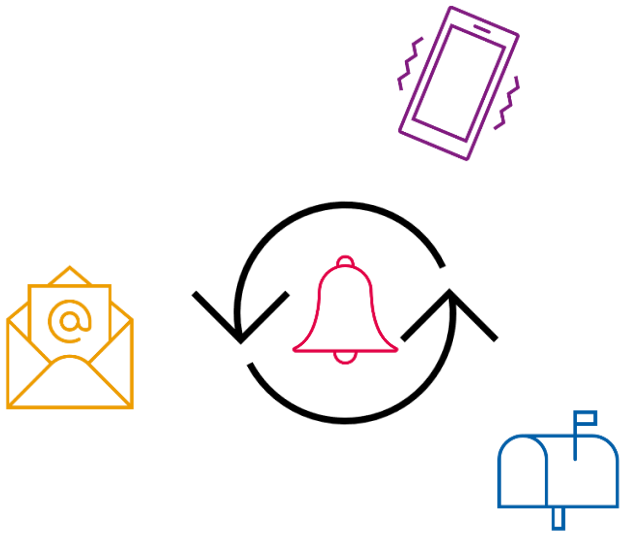


Polymorphism

Definition

- Polymorphism refers to the ability to implement inherited properties or methods in different ways across multiple abstractions.

Example



Key Points

- Polymorphism enables flexibility and extensibility in code by allowing methods to behave differently based on the object calling them.

Polymorphism

```
public class Shape {
    public virtual void Draw() {
        Console.WriteLine("Drawing a shape");
    }
}

public class Circle : Shape {
    public override void Draw() {
        Console.WriteLine("Drawing a circle");
    }
}

public class Rectangle : Shape {
    public override void Draw() {
        Console.WriteLine("Drawing a rectangle");
    }
}
```

OOP Languages

Object-Oriented Programming (OOP) Languages

- C#
- C++
- Java
- Python

etc.

Non-Object-Oriented Languages

- JavaScript
- C

etc.

OOP Languages

Object-Oriented Programming (OOP) Languages

- C#
- C++
- Java
- Python

etc.

Non-Object-Oriented Languages

- JavaScript
- C

etc.

Inheritance and polymorphism Knowledge check

- https://learn.microsoft.com/en-us/training/paths/implement-inheritance-polymorphism/
- Units:
  - 3 - Configure base and derived classes
  - 8 - Module assessment